Tutorial: from zero to a running beacon

Contents

Start with nothing. Finish with a GA4GH Beacon v2 service running on your machine, holding data, answering genomic queries correctly.

Time: about 15 minutes, most of it waiting for a Docker build.

Every command and every output on this page was run end to end on 2026-08-19. Where output is shown, that is what was actually printed — not what should be printed. If yours differs, something is wrong and the

troubleshooting section at the end probably covers it.

> Deploying to a server instead? This tutorial covers a local instance. For a public deployment on a VM with TLS and a tunnel, see PR #9 (`docs/DEPLOYMENT_TUTORIAL.md`), written from a real production deploy. It is not merged yet, so read it from the pull request.

1 What you will have at the end

- MongoDB 5, Redis 6 and the Django API running in Docker
- 100 variants, 50 individuals and 1 dataset loaded
- A beacon that answers `exists: true` for a variant it holds and `false` for one it does not
- Enough understanding of the query path to change it safely

2 Before you start

You need Docker with Compose v2, and about 2 GB of free disk.

```
docker --version
docker info >/dev/null && echo "daemon is running"
```

2.1 On Windows

Every command on this page is written for a POSIX shell — bash or zsh, as on macOS and Linux. Two things break in PowerShell:

- Line continuation. These pages end a wrapped command with a backslash. PowerShell uses a backtick (`` ` ``) instead, so it treats the backslash line as a complete command and fails. Either join the command onto one line, or swap the trailing `\` for `` ` ``.
- **`sed`** does not exist. Step 2 uses it to edit one line of a config file. Edit that file in a text editor instead.

The smoother path is WSL2. Docker Desktop on Windows already runs a WSL2 backend, so opening a WSL shell costs nothing and every command here then works exactly as written, with no translation. Run `wsl` and clone the repository inside the Linux filesystem — not under `/mnt/c`, where Docker bind mounts are slow.

If you would rather stay in PowerShell, the joining rule above is the whole translation. For example, step 6's check becomes one line:

```
docker compose -f compose/docker-compose.dev.yml exec -T mongodb mongo beacon_db --quiet
 ↪  --eval 'db.getCollectionNames().forEach(function(c){ print("  "+c+": "+db[c].count())
 ↪  })'
```

Verified against Docker 29.2.1. You do not need Python, MongoDB or Node installed locally — everything runs in containers. You will want Python later to run the test suites, but not for this tutorial.

---

3 Step 1 — Get the code

```
git clone https://github.com/AfriGen-D/variant-checker-beacon.git
cd variant-checker-beacon
```

Use the HTTPS URL above unless you have already set up an SSH key with GitHub. The repository is public, so HTTPS needs no account at all. The SSH form, `git@github.com:AfriGen-D/variant-checker-`

beacon.git, authenticates even for public repositories, and without a key on your GitHub account it fails with

```
ERROR: Repository not found.
fatal: Could not read from remote repository.
Please make sure you have the correct access rights and the repository exists.
```

which reads as though the repository is missing or private. It is neither — that is an authentication failure wearing a misleading message.

The directory is **variant-checker-beacon**. The repository was renamed from afrigen-beacon-v2; git clone names the directory after the repository, not after the old name.

Clone somewhere Docker Desktop can share — under your home directory is safe. On macOS, /tmp is not in Docker Desktop's shared paths by default, and bind mounts from there silently resolve to empty directories inside the VM instead of failing. That produces a container that cannot write its log file and an API that never starts, with an error that points nowhere near the cause. This cost an hour to diagnose while writing this page.

4 Step 2 — Check the environment file exists

The dev stack reads .env.boolean from the repository root. It is gitignored (**.gitignore:62**), so a fresh clone does not have it and docker compose will refuse to start:

```
env file /path/to/repo/.env.boolean not found
```

Create it from the example:

```
cp .env.example .env.boolean
```

Then make one edit, or nothing will work. .env.example is a production template: it sets SECURE_SSL_REDIRECT=True, which makes Django 301-redirect every request to https://localhost:8000 — where nothing is listening. You get a stack that looks healthy in docker ps and answers nothing.

On macOS:

```
sed -i '' 's/^SECURE_SSL_REDIRECT=True/SECURE_SSL_REDIRECT=False/' .env.boolean
```

On GNU/Linux, without the empty argument after -i:

```
sed -i 's/^SECURE_SSL_REDIRECT=True/SECURE_SSL_REDIRECT=False/' .env.boolean
```

On Windows PowerShell, sed does not exist:

```
(Get-Content .env.boolean) -replace
 ↪  '^SECURE_SSL_REDIRECT=True','SECURE_SSL_REDIRECT=False' | Set-Content .env.boolean
```

DJANGO_SECRET_KEY ships as CHANGE_ME, which is fine locally. Never reuse it anywhere real.

> If you change .env.boolean later, docker compose restart will not pick it up — env_file is read when the container is created. Use docker compose -f compose/docker-compose.dev.yml up -d --force-recreate beacon-api.

5 Step 3 — Start the stack

```
docker compose -f compose/docker-compose.dev.yml up -d --build mongodb redis beacon-api
```

The first run builds the API image and takes a few minutes. Compose starts MongoDB and Redis first and waits for both to report healthy before starting the API, so you should not see connection errors during startup.

Expected tail:

```
 Container beacon-dev-mongodb  Healthy
 Container beacon-dev-redis    Healthy
 Container beacon-api-boolean  Started
```

Confirm all three are up:

```
docker compose -f compose/docker-compose.dev.yml ps
```

```
NAME                 IMAGE                STATUS                     PORTS
beacon-api-boolean   compose-beacon-api   Up (health: starting)
 ↪  127.0.0.1:8000->8000/tcp
beacon-dev-mongodb   mongo:5.0            Up (healthy)
 ↪  127.0.0.1:27017->27017/tcp
beacon-dev-redis     redis:6-alpine      Up (healthy)            127.0.0.1:6380->6379/tcp
```

Note the ports bind to `127.0.0.1`, not `0.0.0.0` — the stack is not reachable from your network. Redis is on 6380 locally to avoid colliding with any Redis you already run.

5.1 If Step 3 fails with “address pools have been fully subnetted”

```
failed to create network walk_beacon-dev-network: Error response from daemon:
all predefined address pools have been fully subnetted
```

Docker has run out of private subnets, not disk or memory. Each compose project claims one and they are not released until the network is removed. On a machine running several projects you will hit this before you hit any resource limit.

Find an idle network belonging to something you own and remove it:

```
docker network ls
docker network inspect <name> --format '{{.Name}}: {{len .Containers}} attached'
docker network rm <name>
```

Only remove a network the inspect showed as 0 attached, and only one that belongs to something of yours.

Do not reach for `docker network prune`. It removes every unused network on the machine, including ones other people’s stopped stacks will want back.

6 Step 4 — Confirm the API is alive

Give it about ten seconds after the container starts, then:

```
curl -s http://localhost:8000/api/health
```

```
{"status":"healthy","version":"2.0.0-boolean",
 "services":{"database":"healthy","cache":"healthy"}}
```

Both `database` and `cache` must say `healthy`. If either does not, the API started before its dependencies were ready — `docker compose restart beacon-api` and try again.

> A passing health check proves the API can reach MongoDB and Redis. It proves nothing about whether queries return correct answers. That distinction has caused real incidents on this project;

`docs/DEPLOYMENT_PREREQUISITES.md` explains why at length.

7 Step 5 — Query the empty beacon

Before loading anything, ask it something. This matters: you want to see what "no" looks like from a beacon that genuinely holds nothing, so you can tell it apart from a broken one later.

```
curl -s 'http://localhost:8000/api/g_variants?assemblyId=GRCh38&referenceName=1&start=10
↪  00000'
```

```
{"meta": {...},
 "responseSummary": {"exists": false, "numTotalResults": 0},
 "response": {"resultSets": [], "beaconHandovers": [...]}}
```

(Abbreviated — `{...}` marks elided fields, so that block is illustrative rather than literal output. Every block fenced as `json` in this file is real and parses.)

Read that response shape carefully. The answer is at `responseSummary.exists`. There is no top-level `exists` field, and several older documents in this repository show client examples that parse one. Those examples silently report "not found" for everything. See api-contract.md.

8 Step 6 — Load the test data

```
docker compose -f compose/docker-compose.dev.yml exec -T beacon-api \
  python manage.py load_boolean_test_data --settings=beacon_project.settings_boolean
```

```
  - Individuals: 50

You can now test Boolean queries!
```

Check what landed:

```
docker compose -f compose/docker-compose.dev.yml exec -T mongodb \
  mongo beacon_db --quiet --eval 'db.getCollectionNames().forEach(function(c){ print("
  ↪  "+c+": "+db[c].count()) })'
```

```
  datasets: 1
  individuals: 50
  query_logs: 62
  variants: 100
```

The `mongo:5.0` image ships the legacy **`mongo`** shell. `mongosh` does not exist in it and any command using `mongosh` will fail — some older docs in this repo get this wrong.

`query_logs` is the audit trail; it fills up as you query. Client IPs in it are truncated to a /24 before being written, and rows expire on a TTL index. See `beacon_api/privacy.py`.

9 Step 7 — Find a variant that actually exists

Do not guess coordinates. Ask the database what it holds:

```
docker compose -f compose/docker-compose.dev.yml exec -T mongodb mongo beacon_db --quiet
 ↪  --eval 'var v=db.variants.findOne(); print("assembly="+v.assembly_id+"
 ↪  chr="+v.reference_name+"  start="+v.start+"  ref="+v.reference_bases+"
 ↪  alt="+v.alternate_bases)'
```

```
assembly=GRCh38  chr=chr1  start=31959452  ref=T  alt=C
```

Your position will be different, and that is not cosmetic. The loader picks each position with `random.randint(1000000, 50000000)`, so every install holds a different 100 variants. A position copied from this page will almost certainly not exist in your database, and the beacon will correctly answer `exists: false` — which looks exactly like a broken beacon.

So capture it into a shell variable and use that from here on:

```
POS=$(docker compose -f compose/docker-compose.dev.yml exec -T mongodb mongo beacon_db
 ↪  --quiet --eval 'print(db.variants.findOne().start)' | tr -d '\r\n')
echo "$POS"
```

Notice the chromosome is stored as `chr1`, not `1`. Which form ends up in the database depends on the ingest pipeline that wrote it, which is exactly why the query path matches both.

10 Step 8 — Ask the beacon a real question

```
curl -s
 ↪  "http://localhost:8000/api/g_variants?assemblyId=GRCh38&referenceName=1&start=$POS" |
 ↪  python3 -c 'import json,sys; print(json.load(sys.stdin)["responseSummary"])'
```

```
{'exists': True, 'numTotalResults': 0}
```

`numTotalResults` really is 0 here, and that is not a contradiction. On this path the counter reports matching datasets, not matching variants. Read `exists` and never infer absence from the count — see api-contract.md.

Note the double quotes around the URL. `$POS` does not expand inside single quotes.

That is a working beacon. You asked with a bare `1` and it matched data stored as `chr1`.

Now confirm it says no when it should:

```
curl -s
 ↪  'http://localhost:8000/api/g_variants?assemblyId=GRCh38&referenceName=1&start=999' \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["responseSummary"])'
```

```
{'exists': False, 'numTotalResults': 0}
```

A negative control is not optional here. A beacon that answers `true` to everything looks identical to a working one until someone checks.

11 Step 9 — See the vocabulary handling

`hg38` and `GRCh38` are the same genome build in different vocabularies — UCSC and GRC. A beacon must answer both identically:

```
for a in GRCh38 hg38 HG38; do
  printf '%-8s ' "$a"
  curl -s "http://localhost:8000/api/g_variants?assemblyId=$a&referenceName=1&start=$POS"
   ↪  \
    | python3 -c 'import json,sys;
     ↪  print(json.load(sys.stdin)["responseSummary"]["exists"])'
done
```

```
GRCh38   True
hg38     True
HG38     True
```

And an assembly the beacon cannot answer for is refused rather than answered:

```
curl -s
 ↪  "http://localhost:8000/api/g_variants?assemblyId=GRCh99&referenceName=1&start=$POS"
```

```
{"error": {"errorCode": 400,
  "errorMessage": "Unknown assembly: GRCh99. Recognised builds: GRCh37, GRCh38. This
   ↪  beacon may not hold data for all of them."}}
```

11.1 Why this step exists

Until 2026-08-19 the middle line read `hg38  False`. The query path compared the caller's literal spelling against what was stored, so a caller using UCSC vocabulary got a confident "no" for a variant the panel was holding. Nobody reported it, because nothing looked broken — a researcher simply concluded the variant was absent from African reference data. It was two clicks away in the UI, because GRCh37 was then one of only two options in the assembly dropdown.

Both halves of that are now closed. `hg38` canonicalises to `GRCh38`, and a build this beacon holds no data for is refused rather than answered:

```
curl -s
 ↪  "http://localhost:8000/api/g_variants?assemblyId=GRCh37&referenceName=1&start=$POS"
```

```
{"error": {"errorCode": 501,
  "errorMessage": "This beacon holds no data for assembly GRCh37, so it cannot answer for
   ↪  that build. This is not a statement about whether the variant exists. Data held:
   ↪  GRCh38."}}
```

A 501 is a signal a client can act on; `exists: false` is an answer it will believe.

Note what the 400 does not say. It lists the builds this beacon recognises, and then explicitly declines to promise it holds data for them. An earlier wording said "this beacon answers for GRCh37, GRCh38" — which sent a caller who mistyped an assembly straight into the 501 above, because GRCh37 is recognised and unheld. `assembly.py` has no access to the dataset catalogue, so rather than claim coverage it cannot verify, it stops claiming.

The assembly dropdown now offers only GRCh38, so a UI user cannot reach this — it is for API callers.

That is the failure mode this project guards hardest against, and the rule it produced is the one to carry into your own changes: when the beacon cannot answer the question you asked, it must refuse — never answer a different question and return 200. `beacon_api/filters.py` is the reference implementation of that judgement and its docstring explains the reasoning; `beacon_api/assembly.py` is the fix that came out of it.

11.2 One more thing to notice

Look closely at a positive answer:

```
{'exists': True, 'numTotalResults': 0}
```

`exists` is `True` while `numTotalResults` is `0`. That is not a typo — the counter reports matching datasets under some paths rather than matching variants. Do not build a client that infers absence from `numTotalResults`.

12 Step 9b — Ask the same question by POST

Beacon v2's own request format nests the parameters. Every client that follows the spec sends this shape, so it is worth seeing it work:

```
curl -s -X POST http://localhost:8000/api/g_variants \
  -H 'Content-Type: application/json' \
  -d '{"query":{"requestParameters":{"assemblyId":"GRCh38","referenceName":"1","start":<
   ↪  POS>}}}'
   ↪  \
  | python3 -c 'import json,sys; print(json.load(sys.stdin)["responseSummary"])'
```

Substitute the <POS> you found in Step 7. It must return exactly what the GET in Step 8 returned — if POST and GET ever disagree, one of them is wrong.

12.1 Why this step exists, and what it nearly shipped

Until 2026-08-20 this returned `400 Invalid characters detected in query`. The sanitizer stringified the nested dict before pattern-matching, and the Python repr's own quote matched its injection pattern — an injection verdict on punctuation the sanitizer introduced itself. GET was unaffected, because flat query strings never produce a repr, which is why nothing looked broken.

Fixing that alone would have been worse than the bug. Nothing in the codebase read `query.requestParameters`, so once the body was let through the query carried no filters at all: an impossible locus returned `exists: true` with `numTotalResults: 100`, identical to an empty body. An honest error would have become a confident YES for a variant that cannot exist.

Both halves shipped together. The lesson is worth more than the fix: a change that unblocks a path is not safe until you check what is behind the path.

13 Step 10 — Look around the rest of the API

Endpoint	What it returns
`/api/`	Beacon info
`/api/datasets`	What is loaded
`/api/entry_types`	Supported entry types
`/api/map`	Endpoint map
`/api/individuals?sex=MALE`	A filtered entity query

```
curl -s http://localhost:8000/api/ | head -c 200
curl -s http://localhost:8000/api/datasets
curl -s http://localhost:8000/api/entry_types
```

```
curl -s http://localhost:8000/api/map
curl -s 'http://localhost:8000/api/individuals?sex=MALE'
```

14 Step 11 — Run the test suites

The query-correctness suites need no dependencies at all — no Django, no database, no network:

```
python3 -m unittest \
  beacon_api.test_query_semantics \
  beacon_api.test_query_injection \
  beacon_api.test_pagination_filters \
  beacon_api.test_assembly \
  beacon_api.test_query_vocabulary \
  beacon_api.test_capabilities \
  beacon_api.test_release \
  beacon_api.test_request_body
```

```
Ran 200 tests in 0.002s

OK
```

That is the merge gate. If you change how a query is interpreted, these are the tests that must stay green — and the ones you extend.

`beacon_api.test_middleware` is different: it imports Django, so it needs a Python between 3.9 and 3.12. Django 4.0 cannot import on 3.13+, which removed the `cgi` module it depends on.

15 Step 12 — Shut down

```
docker compose -f compose/docker-compose.dev.yml down
```

Add `-v` to delete the MongoDB volume as well and start completely fresh next time.

---

16 Troubleshooting

`database: unhealthy` in the health response. The API started before MongoDB finished initialising. `docker compose -f compose/docker-compose.dev.yml restart beacon-api`.

Port 8000, 27017 or 6380 already allocated. Something else is using it. Find it with `lsof -i :8000`, or change the host side of the mapping in `compose/docker-compose.dev.yml`.

`mongosh: not found`. The `mongo:5.0` image only has the legacy `mongo` shell. Use `mongo`.

Every query returns **`exists: false`** after loading data. Almost always a coordinate or vocabulary mismatch rather than a broken beacon. Check three things, in order: are you using the assembly the data was loaded under (Step 7 prints it); is your `start` the stored 0-based value rather than the 1-based VCF position; and does the variant you are asking about actually exist (ask Mongo directly, as in Step 7).

A query changed answers after you edited code. Redis caches responses for five minutes. `docker compose -f compose/docker-compose.dev.yml exec redis redis-cli FLUSHDB`.

Everything returns HTTP 301 / **`curl`** shows **`Location: https://localhost:8000`**. `SECURE_SSL_REDIRECT=True` is still set in `.env.boolean`. See Step 2 — and remember the container must be recreated, not restarted, for an env change to apply.

`ValueError: Unable to configure handler 'file'` in the API logs, container never becomes healthy. The container runs as the non-root user `beacon` (uid 1000, `Dockerfile.boolean:39`) and cannot write `logs/beacon.log`. Almost always means the repository is somewhere Docker Desktop does not share — see Step 1. If you are on a shared path, Docker creates `logs/` with workable ownership automatically and you should not hit this.

Two clones of this repo fight over the same containers. The Compose project name is derived from the directory holding the compose file, which is `compose/` in every clone — so two checkouts both resolve to a project called `compose` and share containers and the MongoDB volume. Pass `-p beacon-myfeature` to `docker compose` in a second checkout, or you will be querying the other one's data without noticing.

The build fails installing Python dependencies. The backend is pinned to Django 4.0.10 by `django-mongoengine`, and needs `setuptools<70`. The Dockerfile handles the ordering; if you are installing by hand outside Docker, install `setuptools<70` first.

17 Where to go next

- architecture.md — how a query becomes an answer
- testing.md — what runs, and what gates a merge
- api-contract.md — the response envelope, and the trap in it
- contributing.md — branching, commits, PRs

Loading real genomic data rather than the test fixture is a separate job: see `docs/ILIFU_DATA_LOADING_GUIDE.md` and `afrigend-beacon2-tools/README.md`. Read the data-tools caveats in testing.md before running an import — there are known issues with loading a second dataset over a first.